

Cloud SDK for Scotty: Ruby Google
Ads implementat...

Cloud SDK for Scotty: Ruby Google Ads implementation

Status: Draft > Review > Current > Needs update > Obsolete

Link: go/cloudsdk-scotty-ruby-ads

Authors: Viacheslav Rostovtsev

Last updated: YYYY-MM-DD

Summary

Scotty (go/scotty, go/scotty-http-protocols) is a shared Google service for handling large HTTP/HTTPS uploads and downloads. Scotty acts as a frontend that offloads the complexity of file transfers from individual product backends. Google API teams using Scotty implement a "customer agent" backend that Scotty notifies about transfer progress and completion, allowing the team's application to manage the file and control access. Cloud SDK Libraries are adding support for Scotty functionality.

This document describes adding support for Scotty in Cloud SDK Ruby Client Libraries. It currently only covers Google Ads "googleads" libraries (aka "Simply") support, which is scoped to **only** uploads in one specific grpc Service.

Scotty end-user data and data flow

There are many types of data that the end-user will or may want to specify in the process of making a Scotty upload. They are:

1. Data object – the binary data the end-user wants to upload.
2. Metadata related to the Data object, divided into:
 - a. Semantic metadata, e.g. video title.
 - b. Operational metadata, e.g. content-type or size.

Semantic metadata will typically be persisted on the server, while operational metadata will only be used during the upload. Some data will be classified differently for different methods, e.g. some APIs might want to persist the size while others will not. Semantic metadata, unlike operational metadata will be part of the request message.

3. Metadata of the initial request – these are typical additional headers that we generate for every method. Authentication headers are also in this category.

Importantly for "googleads" libraries, the end-users will pass a developer key with these headers.

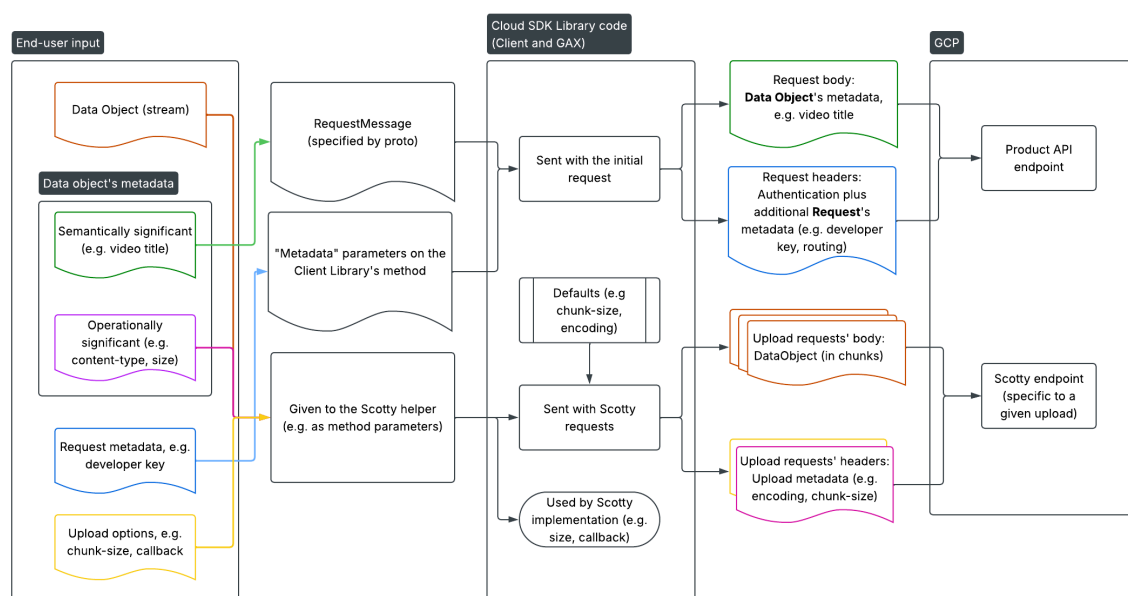
4. Upload options. Includes:
 - a. Retry options and deadline
 - b. End-user requested chunk size (this can be adjusted depending on the server response).
 - c. Callback for progress notifications
 - d. Logger

All of these are optional, with reasonable defaults provided for deadline and chunk size.

All these types of data are sent to either API endpoint (with the initial request) or the Scotty endpoint (with the chunked upload requests).

The Scotty data flow, high-level view

([see in a separate tab](#))



Scotty helper

To help the end-user provide all this data and options in an organized fashion and to allow for future work, I am going to add a "Scotty upload helper" to the gapic-common library. The Scotty upload helper will be a separate class, an instance of which is returned from Scotty upload methods (methods generated for Scotty upload RPCs). The Scotty upload helper will contain instance-level functionality, such as setters and methods to set various upload-related data, e.g.

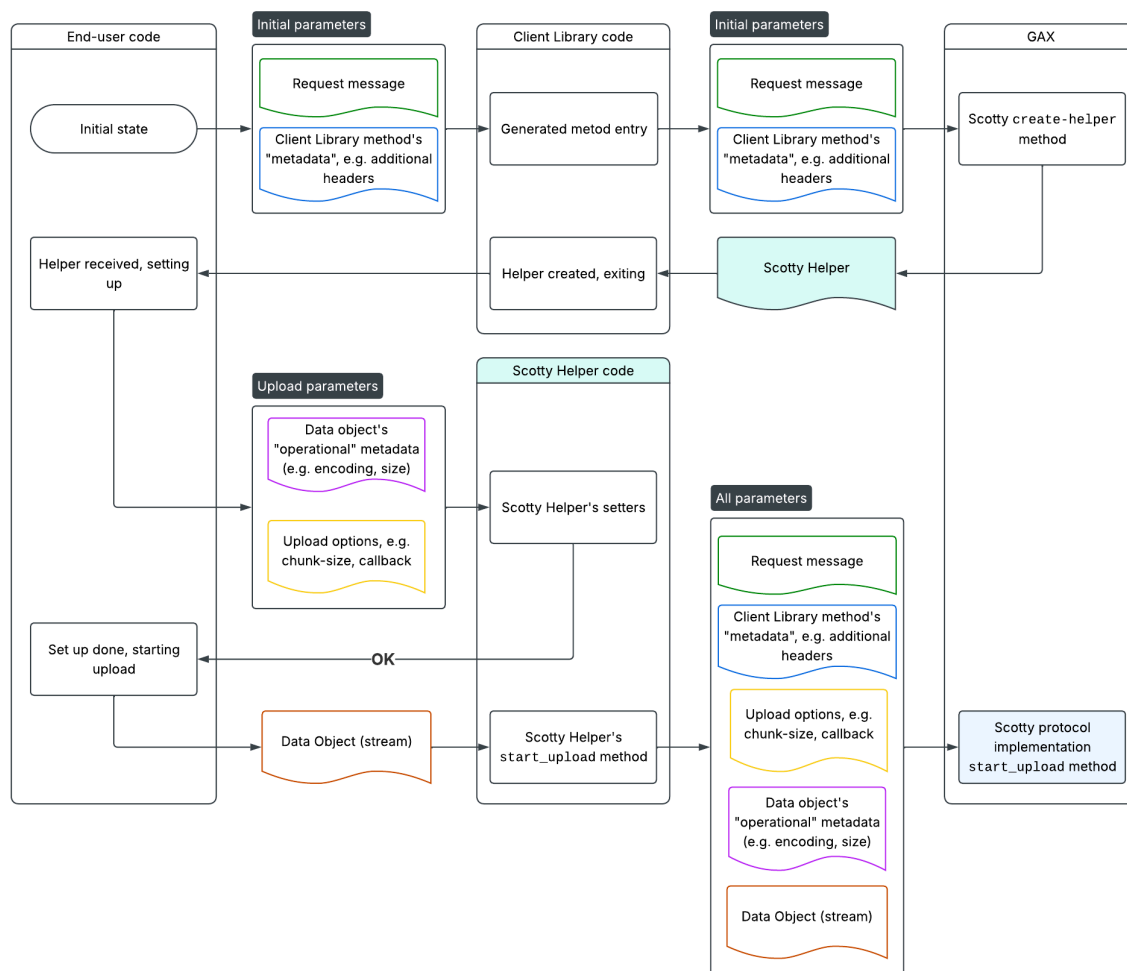
the chunk size. It will also contain the "start-upload" method, called `start_upload` which will take the stream of the data object and trigger the upload exchange.

The interaction between the end-user and the Scotty functionality will be split into three parts:

1. The end-user calls the generated Scotty upload method and receives an initialized Scotty upload helper.
2. The end-user (optionally) interacts with the Scotty upload helper to set the metadata and upload options.
3. The end-user calls the "start-upload" method providing the data stream to upload.

The Scotty upload helper exchange

([see in a separate tab](#))



Protocol implementation

The Unified Resumable Protocol implementation will live in `gpic-common`. To improve readability and testability it will be split into three parts:

The Driver

The only part of the implementation that can do I/O, including reading from the stream, sending HTTP requests, interacting with the `Date` objects.

The Driver makes no decisions. Instead it interacts with the Core, "pumping" it for the instructions, such as "read next chunk", "send next chunk", "send recovery request" etc. As part of this interaction the Driver provides to the core the result of the previous operation, e.g. how many bytes were read from the stream or the server response.

Notable functionality:

- The entry point `start_upload` method.
- All I/O interactions.
- Wrapping socket errors so that they can be retried.
- Transforming server responses without headers into errors so that they can be retried.
- Setting up the `retry_policy` objects.

The Core

The Core keeps track of the protocol state machine current state, such as "are we currently uploading/in recovery/etc". It also keeps track of a small amount of additional information, e.g. how many bytes were uploaded, what was the last offset.

The Core, like the Driver, makes no decisions. Instead it calls the stateless decision function, passing it the result of the previous operation (in the form of `Events`), and receiving two kinds of instructions: to itself, e.g. to update state, and to the Driver, e.g. the next operation to perform.

Notable functionality:

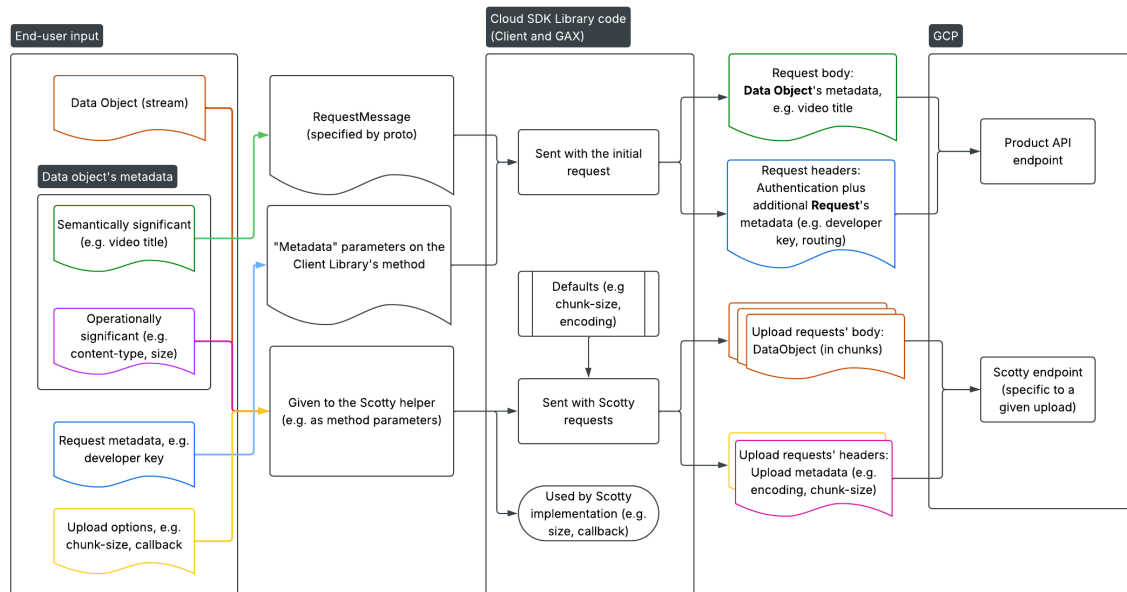
- Mapping from Driver's data to `Events`

Additionally it transforms the information received from the Driver into the `Events` that the Decision function can understand.

1. The Decision function

The Scotty data flow, high-level view

[return to the main document](#)



The Scotty upload helper exchange

[return to the main document](#)

